Rapport de TD n°6 : Initiation à l'algorithmique

Derrez Lorenzo

October 13, 2026

Exercice 0

	début	fin	début	fin	début	fin	début	fin	début	fin	début	fin	début	fin
	Exo 1		Exo 2		Exo 3		Exo 4		Exo 5		Exo 6		Exo 7	
Exo 1	/	/	+	+	+	++	/	/	/	/	/	/	/	/
Exo 2	+	+	/	/	+	+	/	/	/	/	/	/	/	/
Exo 3	+	++	+	+	/	/	/	/	/	/	/	/	/	/
Exo 4	/	/	/	/	/	/	/	/	/	/	/	/	/	/
Exo 5	/	/	/	/	/	/	/	/	/	/	/	/	/	/
Exo 6	/	/	/	/	/	/	/	/	/	/	/	/	/	/
Exo 7	/	/	/	/	/	/	/	/	/	/	/	/	/	/

Exercice 1

Question 1:

- Il faut des entiers long en entrée. Le cours définit le type Longint en Python I1 avec int64.
- **Remarque :** Comme on modifie par effets de bords : Longint, on suppose que son allocation a été effectuée lors du main puis une fois que l'on retourne on libèrera la mémoire avec freeLongint.

Listing 1: Algo1_rec

```
def algo1_rec (n : Longint) -> Longint :
    if n <= 10:
        return 10
    else:
        if (n%2) != 0:
            return algo1_rec(n+3)+algo1_rec(n+1)
        else:
            return algo1_rec(n-2)+algo1_rec(n-7)+algo1_rec(shift(n,-1))
```

Question 2

- Evaluation de la complexité en espace :

  Chacune des variables occupe une place constante

  Il y a des appels récursifs $\implies$ augmente la place occupée sur la pile d'appels $\implies$ augmente la complexité en espace.

  Globalement (avec le shift), on descend de 2 en 2 $\implies$ $\frac{n}{2}$ appels empilés $\implies$ $\#espace = \theta(n)$

- Evaluation de la complexité en temps :

  Au cours de chaque appel, on peut faire :

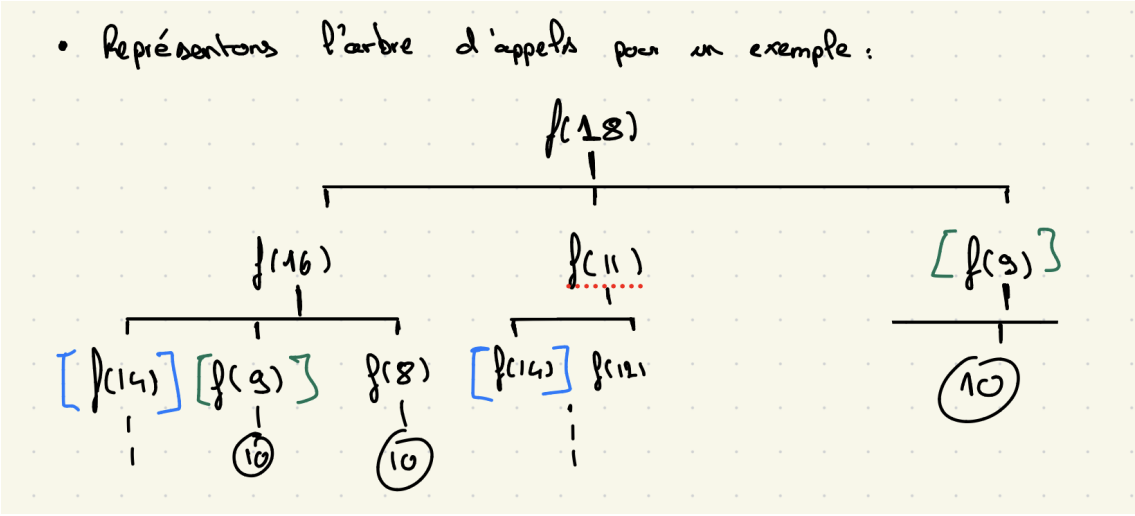
  une comparaison

  un modulo

  des affectations

  des appels récursifs

Pour la complexité en temps : (arbre d'appels)



etc...

- Chaque appel engendre plusieurs sous calculs sur des arguments qui diminuent lentement. (n-2, n-4, etc...)
- Cela entraîne une **complexité en temps exponentielle.**

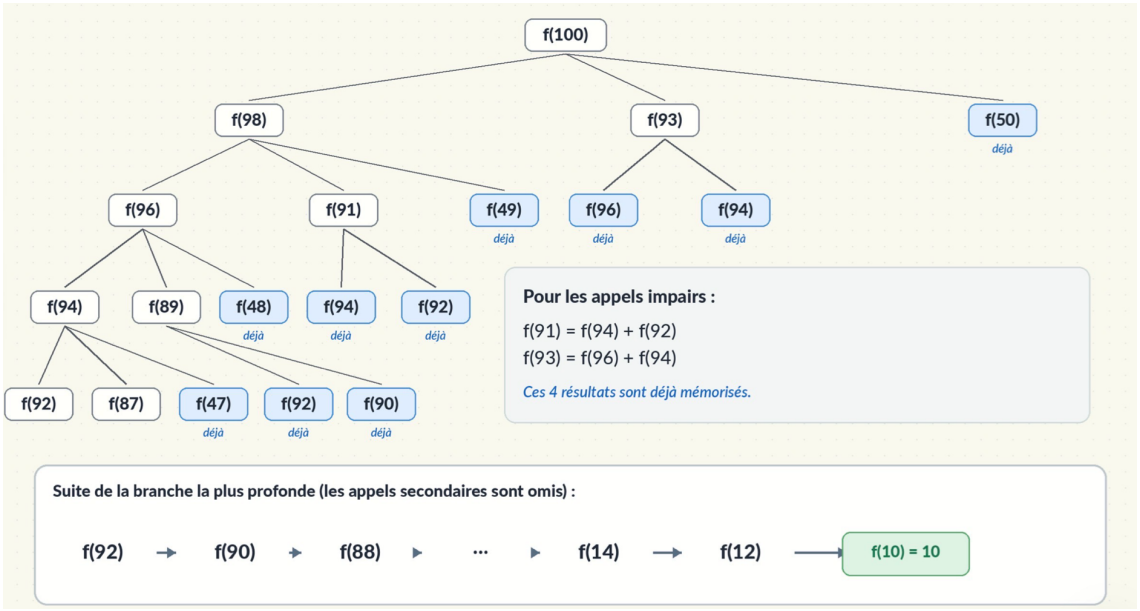
Question 3

- une des solutions pour éviter de calculer une seconde fois une valeur est la **mémoisation**
- memo : tableau de (n+4) initialiée à -1.

Listing 2: Algo2_rec

```
def algo2_rec(memo : Array[integer], n : Longint) -> Longint :
    if memo[n] != -1:
        return memo[n]
    if n<=10:
        resultat = 10
    elif (n%2) != 0:
        resultat = algo2_rec(memo, n+1)+algo2_rec(memo, n+3)
    else:
        resultat = algo2_rec(memo, n-2)+algo2_rec(memo, n-7)+algo2_rec(memo, shift(n,
            -1))
    memo[n] = resultat
    return resultat
```

Question 4



- Complexité en espace :

  $\#f(n) = \theta(n)_{pilememoire} + \theta(n \log n)_{memo} = \theta(n)$

- Complexité en temps :

  $\theta(n)$ malgré les appels récursifs car memo permet de récupérer directemen les valeurs déjà calculées.

Exercice 2

Question 1

- Le problème est syntaxiquement proche du problème *LongeurMaxPlateau*.

Question 2

- Principe :

  tri en $\theta(n \log n) < \theta(n^2)$

  parcours unique en $\theta(n)$

Listing 3: isDiversity

```
def isDiversity(i : integer, j : integer, T : Array[integer]) -> bool :
    S = Array.copy(T[i,j])
    trifusion(S)

    for k in range(1, len(S)):
        if S[k] ==  S[k-1]:
            return false
    return true
```

Question 3

- Evaluation de la complexité temporelle :

  $\#f(n) = \theta(n)_{copy} + \theta(n \log n)_{trifusion} + \theta(n)_{parcoursunique} = \theta(n \log n) < \theta(n^2)$

Question 4

Listing 4: max_variety

```
def max_variety(T: Array[integer]) -> int:
    curr_max = 1
    max = 0
    curr_i = 0
    curr_j = 1

    while curr_i != len(T):
        different = False

        # Verifier que le nouvel element est different de tous les elements du sous-
            tableau courant.
        if curr_j < len(T):
            different = True
            for k in range(curr_i, curr_j):
                if T[k] == T[curr_j]:
                    different = False
                    break

        if different:
            curr_max = curr_max + 1
            curr_j = curr_j + 1
        else:
            if curr_max > max:
                max = curr_max

            curr_i = curr_i + 1
            curr_j = curr_i + 1
            curr_max = 1

    return max
```

Question 5

- Evaluation de la complexité en espace :

  affectation, introduction de variables entières, comparaisons $\implies \theta(1)$

- Evaluation de la complexité en temps :

  Parcours de curr_i à len(T) $\implies \theta(n)$ (où n = len(T) - curr_i)

  Au sein de la boucle dans le pire des cas, on vérifie tous les cas possibles pour chaque indice $\theta(n^2)$ $\implies \theta(n^3)$ au total.

Question 6

Listing 5: *nextIndex*

```
def nextIndex(T):
    n = len(T)
    U = [n] * n

    i = 0
    while i < n:
        for j in range(n - 1, i, -1):
            if T[i] == T[j]:
                U[i] = j
        i += 1
```

```
11
12      return U
```

- **Remarque :** Cette méthode a une complexité temporelle $\theta(n^2)$ et en espace $\theta(n)$.

  Elle n'est peut être pas optimale. Une solution à l'aide d'un dictionnaire pourrait potentiellement éviter les doubles boucles imbriquées. Dès lors on aurait une complexité en $\theta(n)$

Question 7

On considère le tableau de longueur $n = 20$ suivant :

$$T = [1, 2, 1, 3, 4, 2, 5, 6, 3, 7, 4, 8, 5, 2, 6, 1, 7, 3, 8, 4].$$

Son tableau prochain indice est :

$$U = [2, 5, 15, 8, 10, 13, 12, 14, 17, 16, 19, 18, 20, 20, 20, 20, 20, 20, 20, 20].$$

Les indices commencent à 0. La valeur 20 indique qu'il n'existe aucune occurrence suivante. Par exemple, $U[3] = 8$ car $T[3] = T[8] = 3$, sans autre occurrence de 3 entre ces deux indices.

Le sous-tableau

$$T[8 : 16] = [3, 7, 4, 8, 5, 2, 6, 1]$$

contient huit éléments deux à deux différents : c'est donc une diversité de longueur 8.

De plus, T ne contient que huit valeurs distinctes, les entiers de 1 à 8. Aucune diversité ne peut donc avoir une longueur supérieure à 8.

Ainsi, la sortie de la fonction est :

$$\boxed{\text{variety}(T) = 8}.$$

Question 8